

Asymptotic

Exact predicates and Hermitian coefficient arithmetic

An incremental audit for Wolfram Language 15 and Mathics3

Repository <https://github.com/VladimirReshetnikov/Asymptotic>

Reviewed revision efa1aee

efa1aeec4845a9c35e140963a0333d0c9ec33b05

Prepared for Vladimir Reshetnikov

Result. A public conditional-observable call can turn an everywhere-false realness condition into an exact successful result. The failure is reproduced in Wolfram 15, explained by a single admission rule, and accompanied by a conservative candidate repair. A separate, lossless half-spectrum representation for the Fourier coefficient algebra is developed and checked against an independent full-spectrum implementation.

Evidence boundary. Focused Wolfram 15 observations and independent Python tests were executed. Mathics3 execution, a complete upstream test run, and native performance measurements were not available. Existing review findings are not relisted as new defects.

Article, patch-staging tool, regression specifications, exact-algebra prototype, and evidence records are distributed together.

Abstract

This review examines the pinned Asymptotic repository against the substantial existing review corpus rather than producing another inventory of already reported problems. The principal new result is a concrete failure of `inverseFunctionConditionOnJet`: the `Element[... , Reals]` branch tests the retained coefficients of a truncated jet but does not require its remainder to vanish. A cancelled exponential Taylor tail therefore supplies a family of false realness proofs. The public `SeriesObservable` call then returns 1 with zero remainder and `Exact -> True`, although the conditional expression has no germ on the requested positive approach. Five low-cutoff failures and five higher-cutoff refusals were observed on Wolfram Language 15.0.0. A one-line exactness guard rejects the five low-cutoff cases in focused checks while retaining exact polynomial and exact-zero controls. The article also derives a Hermitian half-spectrum implementation for real Fourier-polynomial coefficients, including multiplication, Euler differentiation, the existing inverse-coefficient recurrence, precise operation counts, budget compatibility, and a tested exact Python prototype over $\mathbb{Q}(\sqrt{2})[i]$. Twelve independent Python test methods pass. The report distinguishes observed behavior, mathematical proofs, source-level compatibility analysis, and unexecuted integration work throughout.

Contents

1	Assessment, scope, and exclusion of earlier findings	3
1.1	What this review establishes	3
1.2	Snapshot and source identity	3
1.3	How overlap was checked	4
1.4	Limits of the review	4
2	The representation boundary that matters	5
2.1	A jet is not the function it approximates	5
2.2	The public path	5
2.3	Why the cutoff enters the logical decision	5
3	N01: false realness proofs from cancelled Taylor tails	6
3.1	A minimal public witness	6
3.2	A counterexample family and its proof	6
3.3	The source-level cause	7
3.4	Why two tempting diagnoses are wrong	8
3.5	The quotient-space obstruction	8
4	A narrow repair and its acceptance criteria	8
4.1	The candidate change	8
4.2	What was actually checked after the change	9
4.3	The deliberate conservatism	9
4.4	Regression design that targets the mechanism	10
4.5	Patch staging and replay	10

5	Wolfram 15 and Mathics3: what can be concluded	10
5.1	Wolfram 15 evidence	10
5.2	Source-level Mathics analysis	11
5.3	Version and installation boundaries	11
5.4	A control that did not become a finding	11
6	H01: a lossless half-spectrum coefficient algebra	12
6.1	The current full-spectrum work	12
6.2	Admission must precede compression	12
6.3	Half-spectrum multiplication	13
6.4	Elementary identities that test the difficult cases	13
6.5	Closure under Euler differentiation	14
6.6	Compatibility with the existing inverse recurrence	14
7	Exact work counts, prototype scope, and integration	15
7.1	What is saved, and what is not yet measured	15
7.2	Preserve the meaning of resource limits	15
7.3	The independent exact implementation	16
7.4	Executed test populations	16
7.5	A concrete integration sequence	16
8	Recommended decisions	17
A	Coverage and evidence ledger	17
B	Bundle and reproduction	18
B.1	Contents	18
B.2	Run the independent tests	19
B.3	Stage the source change	19
B.4	Run the public Wolfram-language specifications	19
B.5	Build the article	20

1 Assessment, scope, and exclusion of earlier findings

1.1 What this review establishes

The most important repair identified here is small: an asymptotic approximation must not be used as an exact membership predicate merely because all its displayed coefficients are real. This is not a dispute about the sharpness of a remainder. It is a successful public result for an expression whose stated condition is false everywhere on the approach. The affected path is in the condition checker, before the observable body is constructed. Its body can be the constant 1; no difficult inverse, special function, branch point, or approximate arithmetic is needed.[6, 5]

The distinction between the two contributions in this article is essential. **N01 is a correctness defect with a reproduced public witness. H01 is a concrete implementation proposal with independent algebraic validation, not a measured defect or a claimed native speedup.** They have different acceptance criteria and should not be merged into one numerical “issues found” score.

ID	Classification	Result	Evidence
N01	High-priority correctness	A finite jet is accepted as an exact proof of realness in a conditional observable.	Public Wolfram 15 witnesses; proof; focused candidate-patch checks.
H01	Performance and development proposal	Store only the nonnegative half of a Hermitian Fourier-polynomial spectrum without discarding any mathematical modes.	Exact operation-count derivation; independent Python implementation and tests.

The supporting development work is deliberately specific: regression generation for the predicate failure, a narrow source patch with drift checks, exact half-spectrum admission and closure rules, and a transition plan that preserves the current Fourier resource semantics. General recommendations already present in the retained reviews are not presented again as discoveries.

1.2 Snapshot and source identity

All repository statements refer to commit `efalaeec4845a9c35e140963a0333d0c9ec33b05`. The root standalone is `AsymptoticAnalysis.wl`; the relevant canonical modules are under `src/Kernel`. The article’s repository links are commit-pinned. This matters because the repository has recently incorporated many findings, and an observation from an older report is not evidence that the same failure survives in this revision.[1, 2, 3]

A successful native load was performed by importing the pinned standalone as text, writing it to a temporary file, and loading that file. The imported source contained 692,380 characters and passed `SyntaxQ`. The kernel reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The basic package-only control

```
AsymptoticExpansion[Exp[x], {x, 0, 3}, "Backend" → "Package"]
```

returned the finite expression $1 + x + x^2/2$, as expected for the exclusive cutoff used in that call. The local evidence record is a transcription of successful connector responses, not an upstream TestReport.

One earlier direct Get from the remote URL reported an unexpected end of file. The complete text subsequently passed the syntax check and loaded successfully through the temporary-file route. That event is classified as a transfer/load-path observation, *not* a repository syntax defect. Several other remote calls failed at the connector or network layer; none contributes a package test result.

1.3 How overlap was checked

The current review index describes 45 retained packages distributed over six waves, alongside retired-package records. All six wave indexes and the top-level index were read, as were substantial portions of the maintained status register. Selected nearest original discussions were read rather than relying only on their titles. A separate lexical pass successfully imported all 72 TeX files found under the review tree, including split article sections and reference files.[2, 3]

The lexical scan included the exact helper name `inverseFunctionConditionOnJet`. Other targets were realness/reality-predicate phrases, truncation near predicate terminology, *Hermitian*, and *half-spectrum*. No common-target matches were returned. The initial wave-one pass also used broader cancellation terminology; its extra match concerned ordinary held arithmetic rather than either contribution here. The scan population was 27, 12, 8, 8, 7, and 10 TeX files in waves one through six respectively. The machine-readable record retains the scope and limitations.

This is *not* a claim that every earlier article was reread semantically from beginning to end, or that a line-local regular expression proves conceptual absence. Alternate terminology and line breaks can evade lexical matching. The stronger check is the mechanism-level comparison: the earlier uncertain-trigonometric-error concern concerns an unknown input error; the new witness has the *exact* input x and fails while validating an `Element` condition. The earlier one-sided-observable work concerns the coefficient oracle and limiting point value. The later conditional-observable fast-path finding concerns paths that bypass condition handling. The witness below uses the constant body 1 and enters the condition checker that is supposed to prevent such a result.[12, 13, 14]

Similarly, Hermitian symmetry is not new mathematics, and coefficient-algebra development is not a new project goal. H01's contribution is the concrete lossless representation and convolution schedule, its exact count model, its closure under the repository's existing Euler recurrence, and the independently tested artifact. The proposal does not restate generic advice to add caching, improve sparse multiplication, or create a shared coefficient registry.

1.4 Limits of the review

Close inspection covered the source paths listed in Appendix A. It included ordinary jets and operations, conditional and inverse-function entry, Fourier coefficients, composite error transport, Gamma-related operations, Dirichlet bounds, certificate paths, incremental inverse helpers, and Mathics adapters. It was not a line-by-line audit of every module, notebook, test, document, or vendored reference.

No Mathics3 kernel was executed. No complete upstream suite was run. No native half-spectrum implementation was substituted into the package. The delivered Wolfram regression file is an executable desired-behavior specification; focused equivalent cases, rather than that complete file as a suite, were executed. These limits prevent a whole-package compatibility or release-readiness claim, but do not weaken the explicit mathematical counterexample or the successful native observations that establish N01.

2 The representation boundary that matters

2.1 A jet is not the function it approximates

The ordinary internal jet has the form

$$J = (T, P, D), \quad T = \{(\beta_j, Q_j)\}_{j=1}^m, \quad (1)$$

with the intended meaning, for a positive local coordinate $w \rightarrow 0^+$,

$$f(w) = \sum_j w^{\beta_j} Q_j(\log w) + \mathcal{O}\left(w^P (1 + |\log w|)^D\right). \quad (2)$$

The rows retain an ordered finite set of weights and polynomial logarithmic coefficients. `P == Infinity` is the ordinary exact-jet convention; a finite P retains an unknown tail. The surrounding series representation can also carry an exact offset and prefactor, so the finite jet is only one part of the public object.[4, 5]

Several elementary implications follow directly from this meaning. An empty row list with finite P is a pure remainder, not exact zero. A real polynomial in the row list describes the finite approximation, not automatically the entire remainder class. An expression's numerical smallness says nothing by itself about exact membership in $\mathbb{R}$, the integers, or an algebraic variety.[16] In particular,

$$0 \quad \text{and} \quad iw^P \quad (3)$$

share the same zero jet modulo $\mathcal{O}(w^P)$, but only the first is real for positive w .

The package's arithmetic can correctly preserve an error bound and still be misused by a caller that reads only `j[[1]]`. `N01` is precisely such a caller: it asks the jet algebra for an approximation to the *argument of a predicate*, then treats the coefficient list as a proof of the predicate itself.

2.2 The public path

The ordinary `SeriesObservable` route first acquires and flattens a series representation. It resolves an operation cutoff, peels a surrounding `ConditionalExpression`, asks `inverseFunctionConditionOnJet` whether the condition is established, and only then applies `seriesJetApply` to the observable body. The result is built by `seriesMake`. [5, 6]

For the witness in the next section, the observable body is exactly 1. Once admission succeeds, constructing this body is trivial and legitimately yields an exact constant *as an algebraic body*. Final real-coefficient checks on that constant cannot recover the condition that was wrongly admitted. This explains why the resulting object can look exceptionally clean: its finite expression is real, its error is zero, and its exactness flag is true. The corruption occurred in the logical gate, not in the final constant's arithmetic.

Wolfram's documented `ConditionalExpression[expr, cond]` denotes `expr` when the condition holds; a false condition evaluates to `Undefined`. Thus removing an everywhere-false condition is not an innocuous simplification of presentation. It changes an undefined conditional expression into an ordinary value.[15]

2.3 Why the cutoff enters the logical decision

The same requested cutoff h used for an observable calculation is passed to the condition-on-jet routine. This can be appropriate for a *sufficient* asymptotic sign proof: a retained positive leading term may dominate a controlled smaller remainder. It is not sufficient for exact realness merely to fail to see a nonreal term below h .

A useful distinction is between a proof being unavailable at low precision and a proof being false. A low-precision checker may return “not proved” and subsequently establish a condition after refinement. It must not establish a condition at low precision that becomes disproved when one more term is retained. The reproduced cutoff pairs below exhibit exactly that reversal.

3 N01: false realness proofs from cancelled Taylor tails

3.1 A minimal public witness

Load the pinned package, then evaluate the following with otherwise unassigned symbols. The symbolic parameter a carries the assumption $a == I$; the actual source being expanded is the exact real function x .

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  x, {x, 0, 3}, Assumptions → a == I,
  "Backend" → "Package"];

bad = AsymptoticAnalysis`SeriesObservable[
  s,
  ConditionalExpression[
    1, Element[a (Exp[z] - 1 - z), Reals]],
  z, "Cutoff" → 2];

{bad["Expression"], bad["Remainder"], bad["Exact"]}
(* {1, 0, True} on the reviewed baseline *)
```

The source object has expression x , remainder zero, and the retained assumption $a == I$. This is not a request to expand a complex-valued source. A complex quantity appears in a *condition that should be rejected*; treating it as a proved real quantity is the failure.

Increasing the observable cutoff from 2 to 3 changes the response to

```
Failure["IncompatibleObservableCondition", ...]
```

The higher-cutoff refusal is correct. The lower-cutoff success is not a weaker but valid expansion of the same object: the conditional expression has no value on the positive approach at all.

An independent native simplification returned

```
FullSimplify[
  Element[a (Exp[x] - 1 - x), Reals],
  a == I && x > 0]
(* False *)
```

This check supports the observation, but the proof does not depend on a simplifier.

3.2 A counterexample family and its proof

For a nonnegative integer m , define

$$T_m(x) = e^x - \sum_{k=0}^m \frac{x^k}{k!}. \quad (4)$$

Use the condition

$$C_m(x) : \quad aT_m(x) \in \mathbb{R}, \quad a = i. \quad (5)$$

Proposition 1 *For every integer $m \geq 0$, $C_m(x)$ is false for every real $x > 0$. Therefore the conditional observable equal to 1 only when C_m holds has no germ on $x \rightarrow 0^+$.*

Proof. The convergent exponential series gives

$$T_m(x) = \sum_{k=m+1}^{\infty} \frac{x^k}{k!} > 0 \quad (x > 0).$$

Consequently $iT_m(x)$ has zero real part and a strictly positive imaginary part. It is not a real number at any point of the approach. There is no punctured positive interval on which the condition is true. $\square$

The family also exposes exactly why the error is cutoff-sensitive:

$$iT_m(x) = \frac{i}{(m+1)!} x^{m+1} + O(x^{m+2}). \quad (6)$$

An exclusive cutoff $h = m + 1$ omits the first nonreal term. The returned approximation to the predicate argument is a pure remainder. At $h = m + 2$, the first nonreal coefficient becomes visible. The observed native behavior was:

m	Low cutoff	Low-cutoff result	Higher cutoff	Higher-cutoff result
0	1	exact 1	2	condition refused
1	2	exact 1	3	condition refused
2	3	exact 1	4	condition refused
3	4	exact 1	5	condition refused
4	5	exact 1	6	condition refused

All five low-cutoff objects reported expression 1, remainder zero, and Exact $\rightarrow$ True. The five higher-cutoff results used the tag IncompatibleObservableCondition. These ten observations are recorded explicitly; no count is inferred from an unexecuted loop or from the mathematical generalization to all m .

3.3 The source-level cause

The relevant branch is structurally:

```
If[head === Element && c[[2]] === Reals,
  j = seriesJetApply[c[[1]], x, input, d, cut, limit];
  Return[
    And @@ (
      TrueQ[FullSimplify[
        Element[#[[2]], Reals],
        seriesAss[d] && Element[d["LogVariable"], Reals]
      ]] & /@ j[[1]]],
    Module[]];
```

Only $j[[1]]$, the finite rows, participates in the proof. The finite remainder exponent $j[[2]]$ is ignored. When the rows are empty, the conjunction is empty and hence true. More generally, a real finite prefix can hide a nonreal tail; emptiness is the simplest manifestation, not the entire issue.[6]

The neighboring equality handling demonstrates the missing distinction clearly. When a relational difference has no retained rows, that branch requires $j[[2]] === \text{Infinity}$ before using exact equality or the corresponding non-strict relations. The realness branch has no analogous exactness requirement. This observation identifies a local inconsistency; it does not imply that every other relation is fully audited or that copying one check proves the entire predicate engine correct.[6]

3.4 Why two tempting diagnoses are wrong

The failure is not that a high-order nonreal monomial is universally dropped. The control `Element[a z^2, Reals]`, under $a^2 == -1$, was refused at both cutoffs 1 and 3. The exact polynomial path retains the coefficient information needed to reject it. The cancelled exponential tail is significant because the analytic composition introduces finite precision while removing the low-order rows.

Nor is the failure caused by the constant body having an incorrect coefficient. The body really is 1. A numerical check of the returned finite expression, a comparison of its coefficients against the same body, or an assertion that all displayed coefficients are real will all pass. The regression must inspect whether the *conditional call is admitted*, not just the arithmetic of its selected branch.

Finally, the issue is not repaired by merely retaining the original condition as a text label after computing the result. The ordinary public route claims that the condition has been proved on the input germ. A result with an empty actual domain should not be presented as an ordinary successful asymptotic object whose exactness invites downstream arithmetic. The smallest compatible repair is to refuse an unproved condition in the existing place.

3.5 The quotient-space obstruction

It is helpful to state the limitation independently of this particular code. Let q_P map a function to its finite jet modulo $O(x^P)$. A predicate $\mathcal{P}$ can be decided from that jet alone only when the decision is valid for every function consistent with the retained information. Real-valuedness does not descend to this quotient:

$$q_P(0) = q_P(ix^P), \quad \mathcal{P}(0) = \text{true}, \quad \mathcal{P}(ix^P) = \text{false}.$$

Thus no rearrangement of the retained real coefficients can provide the missing proof. Either exactness or additional semantic evidence about the discarded part is necessary.

This observation also prevents an overcorrection. A nonzero magnitude remainder does *not* prove that an expression is nonreal. Both a real and a nonreal function may share it. The appropriate low-information outcome is “not proved,” which the existing public call can express through `IncompatibleObservableCondition`. The candidate patch below does not label all finite tails as complex.

4 A narrow repair and its acceptance criteria

4.1 The candidate change

For the current ordinary-jet convention, the minimal conservative repair is:

```
Return[
  j[[2]] === Infinity &&
  And @@ (TrueQ[FullSimplify[
    Element[#[[2]], Reals],
    seriesAss[d] && Element[d["LogVariable"], Reals]
  ]] & /@ j[[1]]),
  Module]
```

It adds one requirement: a coefficient-only realness proof is permitted only for an exact jet. An empty exact jet still proves exact zero and hence realness. An empty finite-precision jet no longer does. Real finite exact polynomials remain admissible when their coefficients are proved real under the retained assumptions.

The patch does not alter `seriesJetApply`, change coefficient arithmetic, invent a new remainder type, or change the public failure tag. It does not affect the mathematical meaning of native `Element`. The changed rule is solely the package's use of its approximation as evidence.

4.2 What was actually checked after the change

The exact original anchor occurred once in the downloaded pinned standalone. Replacing that anchor with the guard above produced text passing `SyntaxQ`. The modified standalone was loaded in Wolfram 15. The following focused checks returned successfully:

Check	Observed candidate behavior
The $m = 1$, cutoff-2 false condition from the minimal witness	<code>IncompatibleObservableCondition</code> .
<code>Element[z^2, Reals]</code> on the same exact input	Exact constant body: expression 1, remainder 0.
All five family members $m = 0, \dots, 4$ at cutoff $m + 1$	Every call refused with the expected condition tag.
The same cancelled-tail condition applied to an exact zero source	Expression 1, remainder 0, <code>Exact</code> $\rightarrow$ <code>True</code> .

The first family member check in the focused call and the subsequent family call overlap at $m = 1$; they are not counted as two distinct regressions. The evidence file preserves both observations rather than manufacturing a suite total.

These checks validate the modified standalone route, not a complete modular rebuild. The source-patch tool and the supplied regression file still need to be applied and run in a real checkout before integration. The complete upstream suite and Mathics runtime were not executed by this audit.

4.3 The deliberate conservatism

The guard is sufficient for this coefficient-only proof rule, but not complete for all real expressions. A non-polynomial expression can be genuinely real on a real input while its truncated jet has a finite remainder. Rejecting such a jet as insufficient *coefficient-only evidence* is conservative. It does not say that the original expression is nonreal.

Consequently the patch is best understood as closing an unsound implication, not as completing real-domain inference. A later extension may establish an individual complete predicate from an exact source expression and the already retained input domain. Such an extension must prove the actual composed argument, not replace it by the finite approximation and reuse the same invalid implication. This report does not propose a second broad real-germ architecture under a new name; those wider obligations already have entries in the existing review register.[3]

The immediate tradeoff is therefore explicit: retain the established exact cases and refuse finite-precision cases that previously succeeded without the required proof. For a correctness-first mathematical API, that is preferable to returning an exact value on an empty domain. A future completeness improvement can restore particular refused cases with a separate proof, without weakening this boundary again.

4.4 Regression design that targets the mechanism

The delivered tests/`PredicateRealness.wl` uses a small explicit runner and does not depend on MUnit. It expects the false-predicate calls to be refused. The unpatched baseline is therefore expected to fail the corresponding desired-behavior assertions. The complete file contains the exact source control, ten cutoff-pair cases, exact real polynomial controls, an exact nonreal polynomial refusal, and an exact-zero control.

The cutoff-pair construction is more discriminating than a collection of unrelated functions. It deliberately moves the first decisive coefficient across the exclusive cutoff while leaving the underlying condition unchanged. It tests whether the checker confuses “not visible” with “proved absent.” The exact-zero control prevents a repair from conflating a genuinely zero expression with a pure remainder. The exact polynomial control prevents the easiest but unnecessarily broad workaround of rejecting every `Element` condition.

A useful expansion of this same regression family varies the body independently from the predicate: use 1, a real constant, and a supported real polynomial body. Their conditional calls should have the same admission decision. This tests that a harmless body cannot mask an invalid precondition. Such additional cases are a proposed extension of this narrow regression, not recorded executed results.

Do not make the regression depend on a high-precision numerical sample of T_m . Near zero, subtracting a Taylor polynomial from an exponential can suffer cancellation, exactly where the analytic proof is simplest. The positivity series proves the required fact without any numerical tolerance.

4.5 Patch staging and replay

The supplied code/`patch_predicate.py` reads a checkout but never modifies it. It requires the pinned commit, compares the relevant working modular file with the committed version, refuses a missing or ambiguous source anchor, and refuses reapplication. It writes a modified modular file and a unified diff to a separate, previously nonexistent output directory.

Four independent fixture tests check the transformation itself: the single edit, a missing anchor, duplicate anchors, and an already patched anchor. They do not claim that an entire checkout was staged locally. Network restrictions prevented obtaining a complete local repository, so the staging command’s real-checkout path was not executed in this audit. Its purpose is to make the proposed edit reproducible and resistant to accidental application to a different revision.

After reviewing and applying the generated diff in a disposable pinned checkout, regenerate the standalone through the repository’s existing builder. That rebuild is necessary because the canonical modular source and the generated public load artifact are different files. No claim is made that editing the modular file alone changes an already downloaded standalone.

5 Wolfram 15 and Mathics3: what can be concluded

5.1 Wolfram 15 evidence

The native observations use Wolfram 15.0.0 on Linux x86-64. They establish that N01 is not merely a hypothetical concern about an alternate interpreter. A fully loaded public package call exhibits the failure on the primary evaluator. The fixed-anchor experiments also establish that the proposed local guard is syntactically valid there and changes the selected public outcomes as intended.

The observations do not establish behavior on every Wolfram 15 maintenance release or operating system. They also do not establish that unrelated routes are unaffected. The patch is small, but smallness is not a

substitute for integration testing; the evidence record deliberately keeps focused observations separate from complete-suite acceptance.

5.2 Source-level Mathics analysis

The package isolates compatibility helpers in `AsymptoticAnalysis`Mathics``. The inspected compatibility source supplies private forms for selected operations such as module returns, lookup and key handling, and matching; other modules supply assumptions, numerical, and Taylor support. The main source is parsed under the appropriate compatibility context rather than globally replacing every system definition in the user's session.[8, 9, 11, 10]

N01 lives above those primitive adapters. Its incorrect implication is expressed in the shared condition-on-jet source: obtain a precision-tracked result, ignore its precision, and conjoin realness claims for the finite rows. The one-line repair uses the already existing ordinary exactness sentinel and standard conditional conjunction. It introduces no new algebraic-number solver, special-function dependency, numerical backend, or unsupported proof command.

That makes the guard a plausible portable change at the source level. It does *not* prove that the identical public witness currently reaches the same branch on Mathics. Evaluation of the conditional expression, assumptions containing a `== I`, analytic expansion, and the private realness adapters may differ. A source-level shared defect can be unreachable for a particular input on one interpreter because an earlier operation refuses the input. Such an earlier refusal would not make the defective implication sound; it would change the public coverage of that input.

The appropriate Mathics check is therefore not just to evaluate `And@@{}` or import the isolated helper. Load the actual pinned or patched package, evaluate the public source constructor, verify that the exact source and assumptions were retained, and then run the same low/high-cutoff pairs. The test file is provided for precisely that end-to-end route. No Mathics pass/fail result is filled in speculatively.

5.3 Version and installation boundaries

At the time of this review, the official PyPI project page lists Mathics3 10.0.1 and a Python requirement of at least 3.11. This identifies a concrete public runtime target; it is not a statement that it was installed locally.[17]

The local artifact tests used Python 3.13.5 and only the standard library. They test the independent exact Fourier algebra and the patch transformation. Their success says nothing by itself about Mathics evaluation rules. Conversely, the mathematical proof of the false condition is independent of both interpreters. Keeping those three levels—mathematics, Python model, and native package execution—separate makes the compatibility assessment useful rather than turning unrelated pass counts into a misleading total.

5.4 A control that did not become a finding

A separate probe examined whether changing the public default backend to "Series" could alter refinement of a previously constructed package-only expansion. For the tested e^x source at cutoff 3, `SeriesRefine[s, 5]` returned the same analytic forward result under that changed default and after restoring the original options: the expression through $x^4/24$ and a `PowerLogRemainder[x, 5, 0]`.

This hypothesis was not promoted to a defect. The result is a control for that selected path, not proof that every possible default interaction is correct. Recording such negative outcomes helps distinguish actual review work from a report that presents every source suspicion as a confirmed issue.

6 H01: a lossless half-spectrum coefficient algebra

6.1 The current full-spectrum work

The Fourier engine represents a coefficient in the logarithmic variable L as a finite sum

$$A(L) = \sum_{\omega \in \Omega} A_{\omega}(L) e^{i\omega L}, \quad A_{\omega} \in \mathbb{C}[L], \quad \Omega \subset \mathbb{R}. \quad (7)$$

The source-power weights remain separate from these frequencies. The implementation normalizes and merges equal frequencies, multiplies coefficient spectra by a full Cartesian convolution, and applies the Euler derivative to each mode through

$$\frac{d}{dL} (A_{\omega}(L) e^{i\omega L}) = (A'_{\omega}(L) + i\omega A_{\omega}(L)) e^{i\omega L}. \quad (8)$$

The constructor also verifies the conjugacy relations required for a real coefficient. These mechanisms already exist; H01 does not propose to rediscover sparse convolution or Euler differentiation.[7]

For an admitted real coefficient, the stored modes satisfy

$$A_{-\omega}(L) = \overline{A_{\omega}(L)}, \quad A_0(L) \in \mathbb{R}[L]. \quad (9)$$

Once this relation is established, storing and multiplying both signed halves is redundant. The alternative representation is

$$A(L) = a_0(L) + 2 \operatorname{Re} \sum_{\alpha > 0} a_{\alpha}(L) e^{i\alpha L}. \quad (10)$$

Only the zero polynomial and positive-frequency polynomials are stored. Negative-frequency information is reconstructed by conjugation; no frequency has been discarded and no approximation has been introduced.

This is a finite Fourier-polynomial representation, not an FFT grid. The frequencies need not be integers, rationally commensurate, or periodic with a shared finite period. The algorithm needs exact addition, subtraction, equality, and sign decisions for frequencies, just as the original exact-frequency algebra does.

6.2 Admission must precede compression

The half representation cannot be used as an excuse to accept arbitrary nonreal input. Given a full spectrum, first perform the existing normalization and real-conjugacy checks. Only then retain its nonnegative half. Simply taking the positive rows of an arbitrary spectrum and declaring the negative rows to be their conjugates would change the represented function.

For example, the single full mode e^{iL} is not the real coefficient $2 \cos L$. Compressing it without an admission check and reconstructing a reflected negative mode would silently perform exactly that change. The independent prototype's `from_full` method rejects non-Hermitian input; the zero mode is separately required to have real polynomial coefficients.

Equal-frequency grouping remains indispensable. A frequency can be zero by exact algebraic equality even when its input expression is not syntactically zero. Production compression must use the package's established exact canonicalization and ordering conventions, not a floating-point comparison or string sign test. The prototype makes this obligation explicit by restricting frequencies to a small exact field rather than claiming to replace the package's arbitrary exact-real support.

6.3 Half-spectrum multiplication

Write

$$A = a_0 + \sum_{\alpha > 0} \left(a_\alpha e^{i\alpha L} + \overline{a_\alpha} e^{-i\alpha L} \right),$$

$$B = b_0 + \sum_{\beta > 0} \left(b_\beta e^{i\beta L} + \overline{b_\beta} e^{-i\beta L} \right).$$

The zero/positive interactions are straightforward: $a_0 b_0$ contributes to zero; $a_0 b_\beta$ and $a_\alpha b_0$ contribute to their positive frequencies. Their reflected partners need not be computed separately.

For each positive pair (α, β) , compute only

$$S = a_\alpha b_\beta, \quad C = a_\alpha \overline{b_\beta}. \quad (11)$$

Add S at $\alpha + \beta$. For the difference mode, use

$$\begin{cases} C \text{ at } \alpha - \beta, & \alpha > \beta, \\ \overline{C} \text{ at } \beta - \alpha, & \alpha < \beta, \\ C + \overline{C} \text{ at } 0, & \alpha = \beta. \end{cases} \quad (12)$$

All contributions landing at the same frequency are added exactly, and identically zero polynomials are removed after collection.

Proposition 2 *Equations (11)–(12), together with the zero/positive interactions, produce exactly the nonnegative half of the full Cartesian product of two Hermitian Fourier-polynomial spectra.*

Proof. Each positive pair supplies four signed products in the full expansion. The positive-positive product is S at $\alpha + \beta$, and the negative-negative product is its conjugate at the opposite frequency. The positive-negative product is C at $\alpha - \beta$; the negative-positive product is its conjugate at $\beta - \alpha$. For nonzero difference, exactly one belongs in the positive half. For zero difference, the two terms coincide in frequency and add to the real polynomial $C + \overline{C}$. The zero-mode interactions have the same conjugate pairing because a_0, b_0 are real. Exhausting these cases exhausts the Cartesian product. $\square$

A negative difference must be reflected *and conjugated*. Reflecting its frequency alone is a subtle wrong implementation: it generally reverses the sign of the sine component. Likewise, the equal-frequency case needs $C + \overline{C}$, not just C , and the zero mode must not be doubled a second time at presentation.

6.4 Elementary identities that test the difficult cases

The identity

$$\sin L \sin 2L = \frac{1}{2} \cos L - \frac{1}{2} \cos 3L$$

exercises a negative difference. In the half representation, the output coefficients are $1/4$ at frequency 1 and $-1/4$ at frequency 3. The factor of two in (10) converts them to the displayed cosine coefficients.

Similarly,

$$\sin^2 L = \frac{1}{2} - \frac{1}{2} \cos 2L$$

exercises a collision at zero: the half spectrum contains the real zero coefficient $1/2$ and positive frequency-2 coefficient $-1/4$. Both identities are exact tests in the supplied prototype. They supplement, rather than replace, the larger Cartesian-oracle tests.

6.5 Closure under Euler differentiation

For a stored positive mode, apply $D_L + i\omega$ to its polynomial. The reflected mode transforms as

$$(D_L - i\omega)\overline{A_\omega} = \overline{(D_L + i\omega)A_\omega}.$$

Thus Euler differentiation preserves the Hermitian relation. The zero polynomial's derivative remains real. Addition, multiplication, and scaling by a proved real scalar preserve it as well. Integer powers can therefore use the existing binary-powering strategy entirely within the half representation.

This closure is particularly well suited to the repository's inverse-coefficient recurrence. There is no need to expand to a full signed spectrum after every derivative or real scalar shift. The boundary conversion can be delayed until a public expression, mode list, or independent full-spectrum comparison is requested. Whether that delayed conversion yields a useful end-to-end speedup still needs measurement.

6.6 Compatibility with the existing inverse recurrence

For one perturbation, consider the normalized source model

$$y = u^p \left(1 + \varepsilon u^\delta B(\log u) \right), \quad p \neq 0, \quad \delta > 0. \quad (13)$$

The marker ε organizes the coefficient calculation. With $z = y^{1/p}$, the observable u^r has coefficients of $z^{r+n\delta}$ given by the existing Euler form

$$C_n(L) = \frac{(-1)^n r}{p^n n!} \prod_{j=1}^{n-1} (D_L + r + n\delta + pj) B(L)^n, \quad n \geq 1, \quad (14)$$

and $C_0 = 1$. This is the one-perturbation specialization of the coefficient construction already present in the Fourier module.[7]

For completeness, it follows by setting $v = u^p$, so the implicit equation becomes

$$y = v + \varepsilon v^{1+\delta/p} B(\log v/p).$$

The Lagrange coefficient for $v^{r/p}$ is

$$\frac{(-1)^n}{n!} \left(\frac{d}{dy} \right)^{n-1} \left[\frac{r}{p} y^{(r+n\delta)/p+n-1} B(\log y/p)^n \right].$$

Each derivative supplies a factor p^{-1} times an Euler operator; collecting the powers of y gives (14). This derivation is used here to verify representation closure, not to claim a new inverse algorithm.

Every scalar shift in (14) is real under the admitted real source data. H01 therefore implements the same recurrence without constructing conjugate duplicate polynomial products. For $p = r = \delta = 1$, the first two corrections are

$$C_1 = -B, \quad C_2 = 2B^2 + BB'.$$

For example, $B(L) = (1 + L) \sin L$ gives

$$u = y - y^2 B(\log y) + y^3 (2B^2 + BB')(\log y) + \dots$$

The prototype tests these low-order identities and compares four higher recurrence depths with a separate full-spectrum implementation. No new convergence theorem or remainder improvement is inferred from changing the coefficient storage.

7 Exact work counts, prototype scope, and integration

7.1 What is saved, and what is not yet measured

Let m_A, m_B be the counts of nonzero positive modes. Let $z_A, z_B \in \{0, 1\}$ indicate whether a nonzero zero mode is present. A direct full-spectrum convolution performs

$$F = (2m_A + z_A)(2m_B + z_B) \quad (15)$$

polynomial multiplications before collection. The half algorithm performs

$$H = 2m_A m_B + z_A m_B + z_B m_A + z_A z_B. \quad (16)$$

Hence

$$F - H = 2m_A m_B + z_A m_B + z_B m_A.$$

Without zero modes, the count is exactly halved. With zero modes, the shared zero-zero product is not duplicated in either algorithm.

The delivered test uses $m_A = 2, m_B = 3, z_A = z_B = 1$. Its exact result is identical under both implementations, while the polynomial-product counters are

$$F = 35, \quad H = 18.$$

This is a reduction in a precisely defined operation count for a matched input, *not* a claim of a 35/18 wall-time speedup in Wolfram or Mathics. Polynomial sizes, conjugation, frequency normalization, symbolic assumptions, collection, and public materialization all contribute to actual time. A scalar count also does not describe peak memory or interpreter allocation behavior.

The half representation stores $m_A + z_A$ polynomial entries instead of $2m_A + z_A$. That is a statement about representation cardinality, not a measured object-byte reduction. A public API that returns the full signed spectrum still produces the same final number of public entries.

7.2 Preserve the meaning of resource limits

The existing Fourier code treats `MaxFrequencies` as a hard budget on exact frequencies, not as permission to truncate modes. It also bounds candidate Cartesian products through `MaxTerms`. A storage optimization must not silently reinterpret either option.[7]

For initial integration, retain the old full signed-frequency count

$$N_{\text{public}} = 2m + z$$

when checking or reporting the frequency budget. Likewise, a compatibility mode can check the virtual full pair count (15) before running the half convolution. That preserves the old preflight decision even though fewer polynomial products are actually executed.

This may appear to leave some performance headroom inaccessible under the old budget, but it cleanly separates two changes: lossless representation optimization and API policy. A later explicit policy may permit the cheaper implementation to handle larger jobs under a redefined work budget. Such a policy change should be visible and tested, not hidden inside a conjugacy optimization.

Stage ordering also matters. A product that exceeds the legacy candidate-pair budget must not start expensive new work merely because its final merged support would be small. Conversely, cancellations at the final merge should still remove frequencies before applying the merged-frequency check where the current code does so. Matching final coefficients alone is insufficient when a public option controls refusal behavior.

7.3 The independent exact implementation

The delivered code `hermitian_fourier.py` is an independent standard-library implementation. Frequencies lie in

$$K = \mathbb{Q}(\sqrt{2}),$$

and polynomial coefficients lie in $K[i]$. Exact field elements are stored as rational pairs $a + b\sqrt{2}$; complex coefficients are pairs of such elements. Polynomial coefficients are immutable tuples. Spectrum maps are normalized by exact addition, and zero polynomial entries are removed.

The restricted field was chosen to exercise non-rational frequencies without depending on a symbolic simplifier. For opposite-sign a and b , the sign of $a + b\sqrt{2}$ is decided by comparing a^2 and $2b^2$, after the competing magnitudes have been made positive. No floating-point approximation is used for ordering or equality. Values such as $2 - \sqrt{2}$, $\sqrt{2}$, and $1 + \sqrt{2}$ produce both difference signs and exact collisions during convolution.

This restriction is also an explicit limitation. The prototype does not implement arbitrary exact-real frequencies, symbolic parameter assumptions, native Wolfram expression normalization, package resource accounting, public result objects, source-weight truncation, or error contracts. It is an exact coefficient-algebra experiment, not a drop-in package replacement. Its `lagrange_single` routine covers one perturbation; the package's multi-index frontiers remain outside this artifact.

The reference implementation uses a genuinely different schedule: expand both spectra and take the ordinary full Cartesian product. It shares primitive exact polynomial operations with the optimized implementation but not the half-spectrum routing logic. Equality against this reference therefore checks the most error-prone part of H01: which product is placed at which signed or zero frequency.

7.4 Executed test populations

The Fourier prototype has eight executed test methods. Within those methods, 80 deterministic randomized cases compare half multiplication against full Cartesian convolution, 12 cases check Euler differentiation and the Leibniz rule, and four recurrence depths compare the half and full coefficient constructions. Additional methods check exact field ordering, admission failures, trigonometric identities, low-order inverse coefficients, and the 35-versus-18 work count.

Four separate methods test the patch transformation on small string fixtures. The combined Python run reports **12 methods passed**, with no failures. The saved run used Python 3.13.5 and completed in 0.886 seconds. That duration is a test-run observation in this environment, not a package benchmark and not an expected runtime guarantee for another machine.

The distinction between methods and subcases is deliberate. The 80, 12, and four subcase populations are not added to native observations or to upstream test counts. No Mathics tests are hidden inside these figures. The earlier Fourier-only log is retained alongside the combined log so the progression of local checks is visible.

7.5 A concrete integration sequence

The first implementation boundary should be private to `FourierCoefficients.wl`. Keep the public coefficient and result schema unchanged. Convert an already normalized, already admitted real full spectrum to a private half spectrum; implement the sum/difference convolution and Euler transform there; materialize the full mode list at existing public boundaries. This avoids using a broader representation refactor as a prerequisite for testing the specific optimization.

The first native comparison should run the old and new coefficient paths on identical admitted inputs and compare the normalized full modes exactly. It should include positive/negative frequency differences, zero collisions, algebraic resonances, polynomial cancellation, zero-mode-only inputs, and nonreal input refusals. The existing public `FourierInverseResidual` route can supply an additional integration check after the coefficient comparison, but it should not be the only oracle: a construction and its checker can share assumptions and code.

Only after exact output and budget behavior agree should timing or allocation measurements guide a decision. Matched cases must return the same coefficients and preserve the same source-weight and remainder metadata. A faster call that silently changes frequency accounting, omits modes, or weakens the result is not the proposed optimization. The independent prototype demonstrates correctness of the algebraic schedule; it does not settle whether native conjugation and canonicalization costs make the optimization worthwhile for the repository's actual workloads.

8 Recommended decisions

The correctness and performance work should remain separate changes. For N01, the immediate decision is whether to accept the conservative exactness guard, followed by the distributed desired-behavior regressions on the actual modular rebuild and both intended evaluators. Its public failure tag can remain unchanged. The one-line repair should not be delayed until a complete semantic realness system is designed.

For H01, the immediate decision is whether a private half-spectrum experiment is worth integrating behind the existing full-spectrum boundary. The exact Python model provides a reproducible reference and a precise cost hypothesis. Native measurements should determine whether to keep it. There is no justification here for promising a fixed speedup, changing the frequency cap, or replacing exact real comparisons with numerical ones.

The most useful additional predicate work is narrowly targeted: extend the Taylor-tail family, vary the harmless observable body, and explicitly distinguish exact zero from a zero finite prefix. That directly attacks the demonstrated implication. The most useful additional Fourier work is similarly concrete: retain Hermitian admission, test zero/difference routing, and preserve legacy budget units while measuring the alternative representation.

The wider repository remains a substantial collection of interacting analytic representations and compatibility adapters. This review does not convert its selected controls into a claim that all remaining paths are correct, nor does it restate already registered work as new obligations. Its deliverable is a reproducible incremental result: one specific unsound public admission rule with a tested local repair, and one exact lossless coefficient optimization with enough mathematics and code to evaluate it independently.

A Coverage and evidence ledger

Source area	Inspection scope
<code>src/Kernel/AsymptoticAnalysis.wl</code>	Selected public, assumptions, exact-number, sparse-jet, forward analytic/power, and local-coordinate ranges.
<code>src/Kernel/SeriesOperations.wl</code>	Representation and arithmetic paths; observable entry; composition, truncation, differentiation, and refinement.

Source area	Inspection scope
src/Kernel/InverseFunctionExpressions.wl	Lines 1–175, including the condition-on-jet branch and relevant public entry.
src/Kernel/InverseFunctionSyntax.wl	Callable parsing, arity checks, and condition extraction.
src/Kernel/FourierCoefficients.wl	Coefficient representation, convolution, Euler recurrence, constructor, residual and coefficient entry points.
src/Kernel/SeriesEnvelopeArithmetic.wl	Composite approach, absolute error transport, powers and unary operations.
src/Kernel/DirichletSpecialFunctions.wl	Zeta and Lerch constructors, coefficient selection, and bounds.
src/Kernel/GammaForward.wl	Product/log normalization and factored forward construction.
src/Kernel/GammaInverseOperations.wl	Powered inverse replay and inherited precision.
src/Kernel/InverseCertificates.wl	Selected interval, source-domain, seed and proof paths.
src/Kernel/IncrementalInverse.wl	Selected frontier, polynomial cache, Newton and grouped helpers.
src/Kernel/MathicsCompatibility.wl, MathicsAssumptions.wl, MathicsNumerical.wl, MathicsTaylor.wl	Private compatibility, assumptions, numerical and defining-Taylor paths.
Loaders, workflow, review indexes and status register	Supporting architecture, distribution and exclusion context; not a whole-repository validation run.

The machine-readable evidence/coverage.json expands the boundary between close inspection, executed work, and unexecuted work. The review deliberately leaves unsupported global assertions out of the ledger: there is no claim of absence of additional defects, full Mathics compatibility, measured native speedup, or complete release acceptance.

B Bundle and reproduction

B.1 Contents

The article source is self-contained and uses no external figures or font files. The relevant bundle paths are:

Path	Purpose
article/asymptotic-review.tex and article/asymptotic-review.pdf	This article and its editable LaTeX source.
code/patch_predicate.py	Guarded staging of the N01 modular source edit; emits a patch without changing the input checkout.
tests/PredicateRealness.wl	Desired-behavior public regressions, intended for both package evaluators.

Path	Purpose
code/observe_predicate.wl	Public characterization script that records results without treating baseline behavior as correct.
code/hermitian_fourier.py	Exact half-spectrum prototype and full-spectrum reference primitives.
tests/test_hermitian.py	Executed exact-algebra checks.
tests/test_predicate_patch.py	Executed patch-anchor fixtures.
code/scan_prior_reviews.wl	Reproducible lexical-scan specification; per-wave equivalent batches were executed.
evidence/native-observations.json	Transcribed successful native observations with explicit non-observations.
evidence/prior-review-scan.json	Exclusion method, successful scan counts, nearest comparisons and limitations.
evidence/findings.json, coverage.json, and test logs	Finding classification, scope, and actual independent test results.

B.2 Run the independent tests

From the bundle root, using Python 3.10 or later:

```
python -m unittest discover -s tests -p "test_*.py" -v
```

No third-party Python package is needed. The Mathics interpreter is a separate dependency and is not installed by this command.

B.3 Stage the source change

Use a disposable checkout of the pinned revision. The output directory must not already exist and must be outside that checkout:

```
python code/patch_predicate.py \
--repository /absolute/path/to/Asymptotic \
--output /absolute/path/to/predicate-patch
```

On PowerShell, put the same arguments on one line or use PowerShell's own continuation syntax rather than the POSIX backslash. Review the generated diff, then in the disposable checkout:

```
git apply /absolute/path/to/predicate-patch/predicate-realness.patch
python validation/build_standalone.py
python validation/build_standalone.py --check
```

The script refuses another revision instead of assuming the anchor's survival implies semantic compatibility. It does not generate checksum files.

B.4 Run the public Wolfram-language specifications

In a fresh evaluator session, load the appropriate actual package and then the regression file:

```
Get["/absolute/path/to/Asymptotic/AsymptoticAnalysis.wl"];
Get["/absolute/path/to/bundle/tests/PredicateRealness.wl"]
```

The regression file prints and returns an association and does not exit the kernel. For baseline characterization rather than desired-contract assertions, load `code/observe_predicate.wl` after the package instead. Record the actual evaluator version and the loaded source revision with the output. This article does not prepopulate results for these complete programs.

B.5 Build the article

From the article directory:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error asymptotic-review.tex
```

The document uses standard LaTeX packages, including New TX text/math, listings, longtable, microtype, and hyperref. Auxiliary compilation files are not required in the distributed ZIP. The PDF embeds rendering resources in the normal way; no separate font files are distributed.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, reviewed repository snapshot. <https://github.com/VladimirReshetnikov/Asymptotic/tree/efalaeec4845a9c35e140963a0333d0c9ec33b05>.
- [2] Asymptotic, retained code-review index and linked wave indexes. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/README.md>.
- [3] Asymptotic, *Code review implementation status*. https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/docs/development/CODE_REVIEW_STATUS.md.
- [4] Asymptotic, modular core, `AsymptoticAnalysis.wl`, especially sparse precision-tracked jets and forward operations. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/AsymptoticAnalysis.wl>.
- [5] Asymptotic, `SeriesOperations.wl`, especially `SeriesObservable`, `seriesData`, and `seriesMake`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/SeriesOperations.wl>.
- [6] Asymptotic, `InverseFunctionExpressions.wl`, `inverseFunctionConditionOnJet`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/InverseFunctionExpressions.wl>.
- [7] Asymptotic, `FourierCoefficients.wl`, coefficient algebra, real-conjugacy test, and inverse recurrence. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/FourierCoefficients.wl>.
- [8] Asymptotic, `MathicsCompatibility.wl`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsCompatibility.wl>.
- [9] Asymptotic, `MathicsAssumptions.wl`. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsAssumptions.wl>.

- [10] Asymptotic, *MathicsNumerical.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsNumerical.wl>.
- [11] Asymptotic, *MathicsTaylor.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/MathicsTaylor.wl>.
- [12] Retained review 4, original finding ledger, especially R01 and its stated evidence boundary. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-1/code-review-4/evidence/findings.json>.
- [13] Retained review 25, article source; one-sided observable and native-order findings. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-3/code-review-25/article/article.tex>.
- [14] Retained review 52, article source; conditional-observable fast-path discussion. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efalaeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/code-review-52/article.tex>.
- [15] Wolfram Research, *ConditionalExpression*, Wolfram Language documentation. Accessed 10 September 2026. <https://reference.wolfram.com/language/ref/ConditionalExpression.html>.
- [16] Wolfram Research, *Element*, Wolfram Language documentation. Accessed 10 September 2026. <https://reference.wolfram.com/language/ref/Element.html>.
- [17] Mathics3 maintainers, *Mathics3*, official PyPI project metadata. Accessed 10 September 2026. <https://pypi.org/project/Mathics3/>.